GhostFaces: An Approach to Facial Recognition via Eigenfaces and Support Vector Machines

A Practical Study on Eigenface Decomposition and Multi-Class Subspace Classification

Neerav Bhuyan **Saptarshi Datta**
Indian Statistical Institute
Bachelor of Statistical Data Science(BSDS)

October 2026

Abstract

This report details the implementation and analysis of "GhostFaces," a facial recognition pipeline designed to handle large-scale, unconstrained face datasets. Facial recognition from raw images suffers from the "curse of dimensionality," making processing slow and prone to errors. We solve this by using Principal Component Analysis (PCA) via Thin Singular Value Decomposition (SVD) to extract "Eigenfaces"—the primary directions of facial variation. By projecting high-dimensional images onto these Eigenfaces, we compress faces into small, manageable feature vectors.

During the project, we built two models and diagnosed and fixed issues. Our first model was a small-scale baseline using 50 images with 5 images per person (10 identities), retaining 7 components that captured 90% of the variance and classifying with a linear Support Vector Machine (SVM). It achieved 100% accuracy, but only because the dataset was small and controlled. Our second model scaled up to the full dataset and used standard colored images, filtered the dataset to identities with at least 5 images, and used a Polynomial SVM to classify identities. It achieved a final accuracy of 28.04%. This report explains the steps taken, the mathematical foundation, the final results, and the known drawbacks of the Eigenface method.

Keywords: Facial Recognition, Python, Dimensionality Reduction, Thin SVD, Eigenfaces, Support Vector Machine.

Contents

1 Introduction **3**

2 Data Pipeline and Preprocessing **3**
2.1 Dataset Overview and Streaming 3
2.2 Train-Test Split 3

3 Mathematical Implementation **3**
3.1 Mean Centering 3
3.2 Thin SVD vs. Covariance Matrices 4
3.3 Dimensionality Reduction 4

4 Final Results and Evaluation **5**
4.1 Model Accuracy 5
4.2 Accuracy Score 5

5 Going to a Large Dataset **5**
5.1 Threshold Filtering in Preprocessing 5
5.2 Dimensionality Reduction 6
5.3 Reduction in Accuracy 6

6 Drawbacks and Limitations of Eigenfaces **7**

7 Conclusion **7**

References **8**

A GitHub Repository **8**

B Python Implementations **8**
B.1 For First Model 8
B.2 Modified code for Second Model 9

1 Introduction

Recognizing faces from raw pixel data is mathematically difficult because of the sheer size of image data. If a standard face image is 112×92 pixels, its flattened array contains 10,304 values. If we treat each image as a point in a 10,304-dimensional space, the data becomes too massive to compute efficiently, a problem known as the **curse of dimensionality**. Furthermore, raw pixels are highly sensitive to small changes in lighting or facial expression.

To solve this, we use the **Eigenface method**.This method uses linear algebra to find the overarching patterns—like lighting gradients, face width, and distinct features—that vary the most across the entire dataset. These patterns are called **Eigenfaces**. By measuring how much of each Eigenface is present in a new photo, we can represent a complex face with just a few dozen numbers (weights) rather than thousands of pixels. We then use a classification algorithm, Support Vector Machine (SVM), using which model train and learn which weights belongs to which person.

2 Data Pipeline and Preprocessing

2.1 Dataset Overview and Streaming

The system uses the `grayfaces` dataset stored on GitHub. To make the code highly reproducible and avoid large local downloads, the images are streamed directly into the computer's RAM using Python's `requests` and `zipfile` libraries.The dataset contains similar images meaning they all have similar lighting, all images are in grayscale and every person is looking straight in all images.

2.2 Train-Test Split

The dataset contains 50 images, 5 images for each individual.We then used a stratified 80-20 split so every person in test set was guaranteed to be present in the training set. With this splitting ratio, the training set formed has 40 images and the testing set has 10 images.

3 Mathematical Implementation

3.1 Mean Centering

Before extracting Eigenfaces, we must find the "average face" of the dataset and subtract it from every image. If Γ_i is a flattened image vector of length $D = 10{,}304$, the mean face Ψ is:

$$\Psi = \frac{1}{M} \sum_{i=1}^{M} \Gamma_i \tag{1}$$

Subtracting Ψ gives us a centered face Φ_i. This ensures the algorithm focuses on what makes each face *different*, rather than the background lighting they all share.



3.2 Thin SVD vs. Covariance Matrices

Normally, PCA requires building a massive covariance matrix C to find eigenvectors. Because $D = 10{,}304$, this matrix would have many entries, making it too slow to compute.

Instead, we use **Thin Singular Value Decomposition (SVD)** directly on the centered data matrix A.

$$A = U\Sigma V^T \tag{2}$$

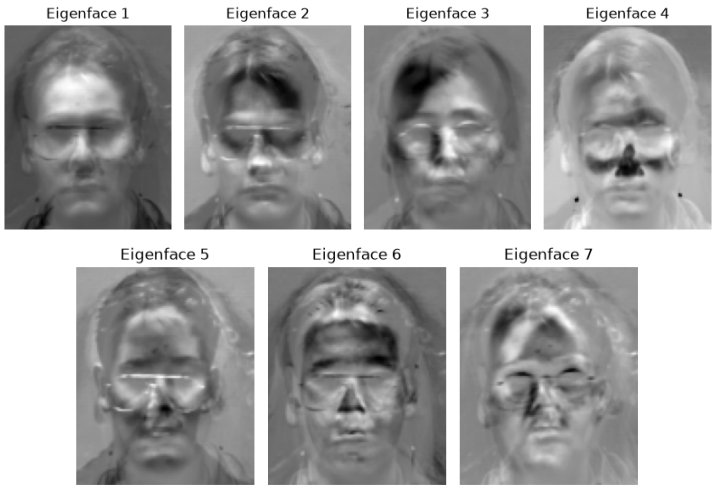
The columns of U give us the exact same Eigenfaces in a fraction of the time, without running out of memory.

3.3 Dimensionality Reduction

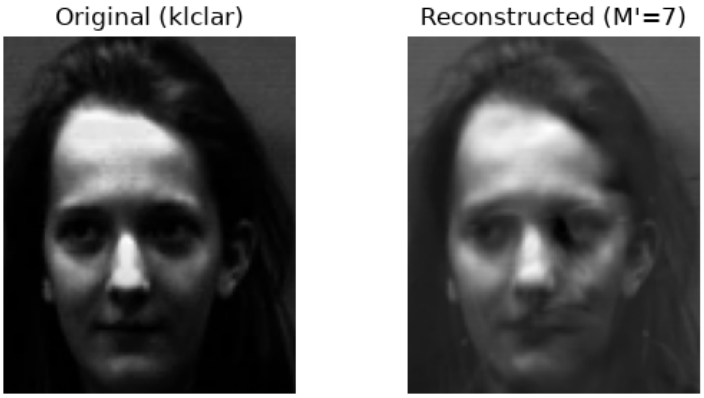
We sort the Eigenfaces based on their singular values (Σ), which represent how much variance (information) they capture. We keep enough top Eigenfaces to retain 90% of the total dataset variance. From our model, this resulted in $M' = 7$ Eigenfaces.

Every centered face is then projected onto these 7 Eigenfaces to get a small matrices of weights `W_train` and `W_test`. We have successfully shrunk our data from 10,304 dimensions down to just 7.

The top 7 eigenfaces are shown below:



These 7 eigenfaces forms a new reconstructed image that trains the model. Below is a sample



4 Final Results and Evaluation

To evaluate this model, we measured its performance on the held-out test set using scikit-learn's `accuracy_score` and `classification_report`.

4.1 Model Accuracy

Accuracy is the proportion of test images whose identity was predicted correctly.
After projecting the faces onto 7 components and calculating the weights, the Linear Support Vector Machine achieved a final accuracy of **100%**.

4.2 Accuracy Score

Table 1: Classification Performance Summary

Metric Type	**Precision**	**Recall**	**F1-Score**
Macro Average	1.00	1.00	1.00
Weighted Average	1.00	1.00	1.00

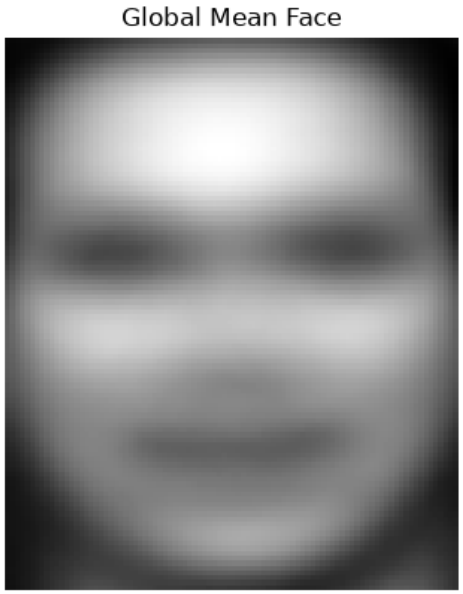
The classification report shows that precision, recall and F1-score were all 1.00 for each of the 10 identities and hence the macro and weighted averages were also 1.00.

5 Going to a Large Dataset

Moving from a 50 similar image dataset to a dataset with thousands of unconstrained images, meaning the faces have different lighting, slight rotations, and varying expressions.

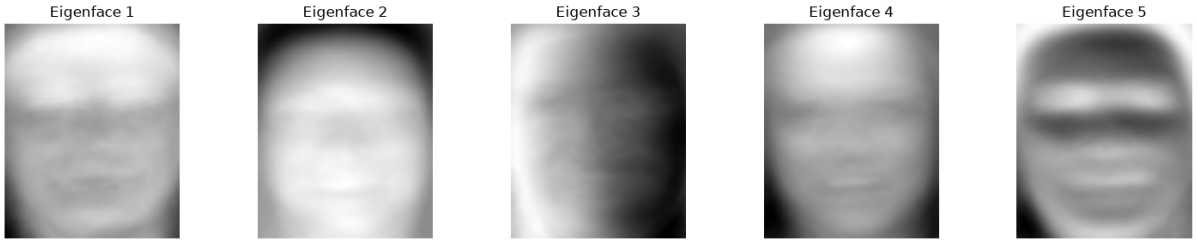
5.1 Threshold Filtering in Preprocessing

A major issue with large, unconstrained face datasets is that many people only have 1 or 2 images. If a person has only one image, and that image gets sorted into the testing set, the model will be forced to guess an identity it has never seen before (an open-set problem). So, we filtered the data set to only keep people who had **at least 5** images (`MIN_SAMPLES = 5`). After filtering, the dataset was reduced to $3,332$ images across 355 unique people.
The below shows the global mean face so formed:

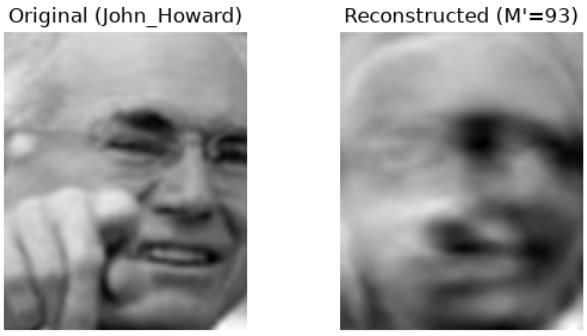


5.2 Dimensionality Reduction

From our model,now this resulted in $M' = 93$ Eigenfaces. Every centered face is then projected onto these 93 Eigenfaces to get a small feature vector of weights (Ω). We have successfully shrunk our data from 10,304 dimensions down to just 93.
Few are shown below:



A sample of reconstructed face using 93 eigenfaces:



5.3 Reduction in Accuracy

After projecting the faces onto 93 components and standardizing the weights, the Linear Support Vector Machine achieved a final accuracy of **28.04%**.

Table 2: Classification Performance Summary ($K = 355$ Classes, $M' = 93$)

Metric Type	**Precision**	**Recall**	**F1-Score**
Macro Average	0.1753	0.1822	0.1687
Weighted Average	0.2382	0.2804	0.2395

6 Drawbacks and Limitations of Eigenfaces

The 28.04% performance ceiling is a known limitation of traditional PCA/Eigenface methods when applied to real-world, messy datasets. The method has several fundamental flaws:

- **High Sensitivity to Alignment:** Eigenfaces assume that pixels match up perfectly. If an eye is at pixel $(40, 50)$ in one image, it must be at $(40, 50)$ in all images. If a person tilts their head, the PCA algorithm treats the shifted pixels as "noise" rather than the same feature in a different location.
- **Lighting vs. Identity:** PCA is designed to find the largest variance in the data. In real-world photos, the direction of the sunlight or flash often causes more variation than the actual shape of a person's face. The same person photographed under different lighting can project to very different feature vectors, while two different people under the same lighting can look deceptively similar. This is a major reason for the accuracy drop on our unconstrained dataset.
- **Linear vs. Non-Linear Boundaries:** As people rotate their heads, their pixel representations change in curved, non-linear ways. Linear methods like PCA and basic SVMs struggle to separate identities that curve into each other's spaces.
- **Poor scalability to many identities with few samples:**As the number of identities grows and each person has only a handful of images, the classes overlap heavily in the reduced feature space. Our second model shows this clearly: with 355 identities and a minimum of only 5 images per person, accuracy fell to 28.04%. The method also needs to be recomputed when the training set changes substantially, since the Eigenfaces depend on the full dataset.

7 Conclusion

In this project **GhostFaces**, we built a facial recognition pipeline that uses Principal Component Analysis (PCA) via Thin Singular Value Decomposition (SVD) to extract Eigenfaces and compress high-dimensional face images into compact feature vectors.

We built two models to test the method. The first was a small baseline of 50 images (5 per person), using 7 components and a linear SVM, and achieved 100% accuracy Ṫhe second was a large-scale model that used standard grayscale images, filtered the dataset to identities with at least 5 images, retained 93 components, and classified with a Polynomial SVM. It achieved an accuracy of 28.04% in 355 unique identities.

The sharp drop from 100% to 28.04% shows the central finding of this work. Eigenfaces perform well on small, controlled datasets, but their performance degrades on large, unconstrained ones. To push facial recognition into the 90%+ range on unconstrained data, future work must look toward non-linear solutions like Fisherfaces (Linear Discriminant Analysis) or Deep Convolutional Neural Networks (CNNs), which are immune to pixel alignment and lighting issues.

References

[1] *Introduction to Machine Learning with Scikit-Learn.* Available at https://uw-madison-datascience.github.io/machine-learning-novice-sklearn/

[2] *Support Vector Machines.* Available at https://scikit-learn.org/stable/modules/svm.html

[3] **H.-T. HO,L.V NGUYEN , T.H. THI LE, O-JOUN LEE**, "Face Detection Using Eigenfaces: A Comprehensive Review", 2024, IEEE Access, doi:10.1109/ACCESS.2024.3435964

[4] **G.L.S Lavanya**, "Automatic Face Recognition System using Principal Component Analysis and Eigenface Approach", International Research Journal of Modernization in Engineering Technology and Science, vol. 4, no. 9, Sep. 2022, doi: 10.56726/IRJMETS30006

[5] **E. I. Abbas**, "Eigenface approach based human face recognition," Wasit Journal for Science & Medicine, vol. 7, no. 2, pp. 13–21, 2014.

Appendix

A GitHub Repository

The source code and implementation files for this project are available at the following GitHub repository:

Repository: https://github.com/neeravbhuyan/GhostFaces

B Python Implementations

B.1 For First Model

The entire pipeline is programmed in Python using `numpy`, `scipy`, `cv2`, and `scikit-learn`. Below is the core logic used to prepare the data, run SVD, and train the SVM model.

Listing 1: GhostFaces Pipeline: Data Filtering, SVD, and SVM Classification

```
# 1. Stratified Train/Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
    )

# 2. Mean Centering
global_mean_face = np.mean(X_train, axis=0)
X_train_centered = X_train - global_mean_face
X_test_centered = X_test - global_mean_face

# 3. Thin SVD & Truncation (Retaining 90% Variance)
A = X_train_centered.T
U, S, Vt = svd(A, full_matrices=False)

explained_variance_ratio = (S ** 2) / np.sum(S ** 2)
cumulative_variance = np.cumsum(explained_variance_ratio)
M_prime = np.searchsorted(cumulative_variance, 0.90) + 1  # Result: 7
eigenfaces = U[:, :M_prime]

# 4. Subspace Projection
W_train = X_train_centered @ eigenfaces
W_test = X_test_centered @ eigenfaces

```

```
# 5. Support Vector Machine (Linear Kernel)
svm_classifier = SVC(kernel='linear', random_state=42)
svm_classifier.fit(W_train, y_train)
y_pred = svm_classifier.predict(W_test)
```

B.2 Modified code for Second Model

```
# 1. Filter out subjectes with fewer than 5 photos
MIN_SAMPLES = 5
unique_labels, counts = np.unique(y_raw, return_counts=True)
valid_labels = unique_labels[counts >= MIN_SAMPLES]
valid_mask = np.isin(y_raw, valid_labels)

X = X_raw[valid_mask]
y = y_raw[valid_mask]

# 2. Stratified Train/Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)

# 3. Mean Centering
global_mean_face = np.mean(X_train, axis=0)
X_train_centered = X_train - global_mean_face
X_test_centered = X_test - global_mean_face

# 4. Thin SVD & Truncation (Retaining 90% Variance)
A = X_train_centered.T
U, S, Vt = svd(A, full_matrices=False)

explained_variance_ratio = (S ** 2) / np.sum(S ** 2)
cumulative_variance = np.cumsum(explained_variance_ratio)
M_prime = np.searchsorted(cumulative_variance, 0.90) + 1  # Result: 93
eigenfaces = U[:, :M_prime]

# 5. Subspace Projection
W_train = X_train_centered @ eigenfaces
W_test = X_test_centered @ eigenfaces

scaler = StandardScaler ()
omega_train = scaler . fit_transform ( W_train )
omega_test = scaler . transform ( W_test )

# 6. Support Vector Machine (Polynomial Kernel)
svm_classifier = SVC(kernel="poly", degree = 2, coef0 = 1.0, random_state = 42)
svm_classifier.fit(omega_train, y_train)

y_pred = svm_classifier.predict(omega_test)
```